Load-Bearing but Redundant: Rethinking Physics Injection in Shared-Latent World Models

Anonymous submission

Abstract

World models for planning should capture physical regularities, but it is unclear whether adding physical structure to a shared visual latent improves long-horizon prediction. We study JEPA-style world models trained without physics supervision: although position is readily decodable and one-step prediction is nearly exact, autoregressive rollouts accumulate error. In a controlled factorial study spanning three physical domains, two training regimes, and realistic simulation benchmarks, we evaluate nine intrinsic injection variants, including hard state slots, supervised probes, kinematic updates, consistency losses, and label-free priors. Injection fails to improve 26 of 27 mechanism–domain combinations, even when the correct gravitational constraint is supplied. Interventions show that the injected slot is *load-bearing but redundant*: prediction uses it, but the remaining latent dimensions already encode the same state. As a positive control, free-rollout training reduces error by 2.2–8.3$\times$ without adding physics. These results suggest that useful inductive biases for shared-latent visual world models should target long-horizon state evolution rather than re-encode state already present in the latent representation.

1 Introduction

A *world model* predicts future states by rolling an internal representation forward under actions. It thereby provides a simulator for planning (Ha and Schmidhuber 2018; LeCun et al. 2022). We study LeWorldModel (LeWM), a JEPA-style world model (Maes et al. 2026), trained on PhyWorld (Kang et al. 2024) without physics supervision. Position is accurately decodable, and one-step prediction is nearly exact (cosine similarity 0.98). Yet collision rollouts fall to 0.24 by horizon 28, and OOD conditions increase error several-fold. Physical state is therefore present in the representation even when the learned transition fails to preserve it over long horizons.

An intuitive remedy is to inject physical structure into the latent representation, as illustrated in Figure 1. This can be done by anchoring selected coordinates to physical state, supervising a physical readout, or imposing a kinematic update. Related approaches have succeeded in two settings. Some route prediction through a low-dimensional physical state by design (Mao, Umasudhan, and Ruchkin 2024). Others apply deep supervision in latent space (Zahorodnii 2025). Evidence is less clear for shared, high-dimensional visual latents, where prior studies use different injection mechanisms and training protocols.

We compare nine injection variants in a common JEPA world model. The variants span hard and soft constraints on physical *state* and its *evolution*, under labeled and label-free supervision. We test them under two training regimes and in three controlled physics domains. We then evaluate the main findings on two realistic-simulation benchmarks (Bear et al. 2021; Tung et al. 2023).

Across this grid, injection fails to improve prediction in 26 of 27 mechanism–domain cells. We also wire the correct gravitational rule, $a{=}g$, directly into the state update. It performs no better than the baseline. Co-training from scratch does not rescue it either: injection stays harmful or at parity in every cell. The result is therefore not specific to adding a constraint after pretraining. The one improvement reverses sign on the other two domains. Using it therefore requires advance knowledge of each domain's dynamics.

The mechanism is redundancy in the latent representation. A pinned slot occupies 2 of the 192 latent dimensions. The other 190 dimensions encode the same position and decode it nearly as accurately as the full latent. The injected copy therefore adds no information that the representation lacks. Its constraint still competes with prediction for capacity and gradient. Redundancy explains the lack of improvement, while this competition can explain degradation.

Interventions confirm that the slot is *load-bearing*. Steering and counterfactual patching both alter prediction well beyond matched controls (§4.3). Yet the slot remains *redundant*: it carries no information that the other dimensions did not already contain.

These observations point to state evolution, rather than state availability, as the main source of long-horizon error. With its horizon matched to the domain, free-rollout training reduces error by 2.2–8.3$\times$. This comparison uses the same code, metrics, and seeds as the injection experiments. This positive control also shows that the evaluation pipeline can detect substantial improvements. An effective inductive bias for a shared latent may therefore need to change how state evolves rather than add another encoding of state. Re-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

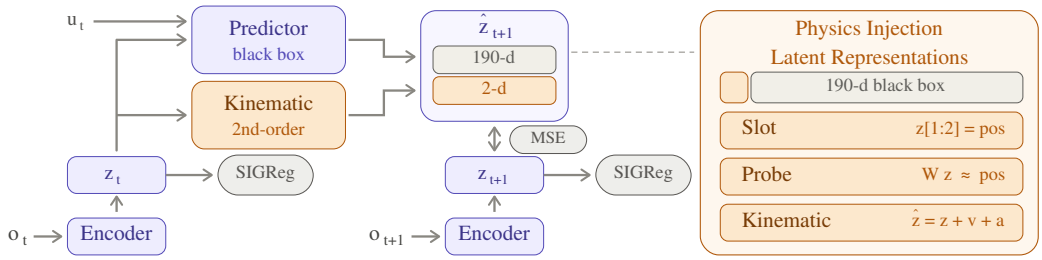


Figure 1: Physics injection into LeWM (*amber* denotes injected components). The encoder maps observation o_t to a 192-d latent z_t; the predictor uses z_t and action u_t to estimate $\hat{z}_{t+1}$, matched by MSE to the encoded target z_{t+1}. Injection reserves a 2-d position slot for a hard pin, supervised probe, or second-order kinematic head. The remaining 190 dimensions retain a separate encoding of the same state, so the slot is causally active but informationally redundant.

weighting a loss cannot remove representational redundancy; doing so requires an architectural change. We port an architecture that makes physical state the *sole* carrier (Mao, Umasudhan, and Ruchkin 2024), which removes the redundancy. However, it collapses when an OOD shift reaches its encoder (ρ 0.33 vs. 0.89). Removing redundancy is therefore not sufficient by itself. Every finding above replicates on a second, frozen-DINOv2 backbone.

Our main contributions are:

- To our knowledge, this is the first systematic study of physics injection in *shared-latent* visual world models. It covers nine variants, two training regimes, three domains, and two realistic-simulation benchmarks. Injection fails to improve on baseline in 26 of 27 cells, and the result replicates on a frozen-DINOv2 backbone (§4.2, §4.5).
- We identify a *load-bearing but redundant* mechanism. Interventions show that prediction uses the injected slot, although the other 190 dimensions retain the same information. Re-weighting reaches at most parity. An extrinsic port that makes physical state the sole carrier isolates the role of the shared-latent architecture (§4.5).
- A controlled factorial identifies training protocol, rather than physical structure, as the dominant variable. Free rollout cuts error by 2.2–8.3× without added physics and provides a positive control against implementation and metric artifacts (§4.4).

2 Related Work

Physically structured world models. Several world-modeling approaches represent physical state explicitly. Physically interpretable models route prediction through a dedicated low-dimensional state (Mao, Umasudhan, and Ruchkin 2024; Peper et al. 2025), and deep supervision improves a low-dimensional (8-D) state latent by adding physical readouts to the objective (Zahorodnii 2025). Phys-JEPA similarly supervises a decomposed JEPA latent and reports gains on numerical time series (Nie et al. 2026). Physics-informed losses (Raissi, Perdikaris, and Karniadakis 2018), conservation-structured dynamics (Greydanus, Dzamba, and Yosinski 2019; Cranmer et al. 2020), and restricted-attention transformers for numerical trajectories (Liu et al. 2026) also operate on representations closely aligned with physical state. Our setting differs because physical variables occupy only a small part of a shared visual latent. We evaluate whether these forms of structure transfer to that setting (§4.2) and use an extrinsic architecture as a control (§4.5).

Latent world models and physics benchmarks. JEPA-style models predict in representation space (Assran et al. 2023; Bardes et al. 2024; Balestriero and LeCun 2025; Maes et al. 2026). Object-level masking changes the JEPA objective but does not explicitly constrain the physical content of the latent (Nam et al. 2026), while DINO-WM plans over frozen visual features (Zhou et al. 2025; Oquab et al. 2024). We use the latter as a cross-backbone control. Dreamer-family models instead train on imagined rollouts (Ha and Schmidhuber 2018; Hafner et al. 2025). For evaluation, PhyWorld provides controlled physics videos with explicit ID and OOD ranges (Kang et al. 2024), and Physion/Physion++ provide more realistic simulated scenes (Bear et al. 2021; Tung et al. 2023). We apply these protocols to latent rather than pixel-space prediction.

Rollout training and exposure bias. Training predictors on their own outputs is a classic remedy for compounding error (Bengio et al. 2015; Venkatraman, Hebert, and Bagnell 2015; Lamb et al. 2016). We claim no novelty for free rollout; its role here is methodological (§3.4): the confound prior injection studies never controlled, and a positive control separating "structure does not help" from "the implementation or metrics are broken."

3 Method: A Design Space of Physical Injection

We organize physical injection into a design space for shared latent representations, specify the intended effect of each variant, and define experiments that distinguish among possible failure mechanisms.

3.1 Setting and Notation

We study LeWM (Maes et al. 2026), a compact JEPA-style action-conditioned world model (∼10M parameters): an encoder E_θ (ViT-tiny) maps frame o_t to a latent $z_t \in \mathbb{R}^D$ with D=192, and a causal-transformer predictor F_ϕ rolls latents forward from the history and action u_t, i.e. $\hat{z}_{t+1} =$

$F_\phi(z_{\leq t}, u_t)$. Training minimizes a prediction loss, an anti-collapse regularizer (Balestriero and LeCun 2025), and—when physics is injected—an auxiliary physical loss:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \beta\,\text{SIGReg}(z) + \lambda\,\mathcal{L}_{\text{phys}}. \tag{1}$$

Here $\mathcal{L}_{\text{pred}} = \frac{1}{D}\sum_i w_i(\hat{z}_{t+1,i} - z_{t+1,i})^2$ with w_i=1 by default; *re-weighting* raises w_i to w on the 2 slot dimensions (w swept $1 \to 300$), while λ scales the probe or consistency loss $\mathcal{L}_{\text{phys}}$ defined in Table 1. These are two distinct dials. *Teacher forcing* trains F_ϕ on one-step transitions from encoded ground-truth context; *free rollout* feeds predictions back for H autoregressive steps (H=8 unless stated) and supervises the whole rollout. The kinematic head (Group 2 below) replaces the slot's transition by a second-order update

$$\hat{z}_{t+1}^{[0:2]} = z_t^{[0:2]} + v_t\,\Delta t + \tfrac{1}{2}\,a\,\Delta t^2, \tag{2}$$

where v_t is the finite-difference slot velocity and the acceleration a is either a learned $\text{MLP}(z_t, u_t)$ or a single learnable constant—the strict a=g head. Unless stated otherwise the encoder is initialized from the public PushT checkpoint; §4.2 additionally studies from-scratch training.

3.2 Injection Design Space: Three Groups, Nine Variants

We study *intrinsic coordinate injection*, which writes physics onto the coordinates of a latent already shared by the predictor. The encoder E_θ and predictor F_ϕ otherwise remain unchanged. This family captures the familiar "just add a physics term" prescription. We vary three orthogonal axes: injecting *what the motion state is* versus *how it evolves*, using *hard versus soft* constraints, and using *labeled versus label-free* supervision. Hard constraints fix latent coordinates to physical quantities, whereas soft constraints add auxiliary losses. The strict a=g variant also serves as the labeled control for the label-free prior. All variants use free-rollout training (§3.4). Table 1 summarizes the resulting variants.

Scope. Two families lie outside intrinsic injection, and our results neither test nor claim to refute them: *object-centric* designs that change what the representation *is* (slots, relational graphs; Locatello et al. 2020; Battaglia et al. 2016; Sanchez-Gonzalez et al. 2020), and *operator-level* designs that replace the predictor's transition function (*Hamiltonian*/symplectic integrators, learned contact; Cranmer et al. 2020). Both alter the architecture rather than annotate a shared latent's coordinates—where §4.3 locates the leverage—so they fall outside the "just add a physics term" prescription we study; we do port one as an external control (§3.5), and bound our claims to intrinsic injection in §5.

We separately vary the *training regime* in a 2×2 design per domain: fine-tuning versus training from scratch, each with injection on or off. This tests whether injection is ineffective only because a pretrained representation has already formed before the constraint is introduced. In the from-scratch condition, the constraint is active from the first update (§4.2).

3.3 Mechanistic Hypothesis and Its Tests

We distinguish two properties of an injected channel. It is *load-bearing* if intervening on it changes the prediction, and *marginal* if it contributes information unavailable in the rest of the latent. A probe establishes neither property (Elazar et al. 2021), and a redundant representation can satisfy the first without satisfying the second.

Variant	What it does, and what it tests
Group 1: inject the state — what the state *is* (position, velocity)	
slot (structpos)	*hard.* pin dims [0:1] to the true position (the object's x, y coordinates; a fixed index range, not an attention slot): the most direct state injection
+reweight	*hard.* raise the slot's share of the prediction loss (w=30, Eq. 1): if failure is due to the slot's ∼1% gradient share, this should rescue it
+velocity	*hard.* encode velocity in dim[2]: tests whether it matters *which* physical quantity is injected
probe	*soft.* linear head reads position from the latent: replicates the deep-supervision recipe (Zahorodnii 2025) in a high-dimensional latent
+slot	*both.* soft + hard combination: tests complementarity
Group 2: inject the evolution — how it *evolves* (a dynamics rule)	
free MLP	*hard.* attach z+v+a update with learned a: upgrade from pinning state to pinning evolution
strict a=g	*hard.* exact gravitational form, one learnable constant: closes the "your physics was wrong" objection. Doubles as Group 3's labeled control (grounded prior)—the two specifications coincide
consistency	*soft.* finite-difference velocity of the rolled-out positions must match ground truth; assumes no form for a, designed for collision impulses
Group 3: drop the labels — is ground truth *needed*?	
label-free	*hard.* the kinematic head is attached but nothing pins the slot to ground truth: the only injection available for raw video. Its labeled control is exactly strict a=g above

Table 1: Nine intrinsic injection variants grouped by target.

Redundancy hypothesis. The physical slot occupies 2 of 192 dimensions, while the remaining dimensions encode the same position. Under this hypothesis, prediction may depend on the slot even though it adds no information to the representation, while its loss still consumes capacity and gradient. We evaluate this account with four tests and state the relevant counter-evidence for each:

1. **Is the physical constraint injected?** Split the latent into the 2 slot dimensions and the 190 black-box dimensions and decode position from each separately, with a random-2-dimension control following the general principle that probes require matched controls (Hewitt and Liang 2019). If the black box alone cannot decode position, the redundancy claim is false.

2. **Is the constraint acting at all?** Measure the weighted physical loss relative to the prediction loss (a gradient-magnitude proxy) and the encoder's effective dimensionality (participation ratio). If neither moves, the failure is a bug; if the term dominates and dimensionality collapses, the constraint is acting—and competing with prediction.
3. **Does re-weighting help?** Sweep the slot weight w from 1 to 300 (Eq. 1). If harm does not respond to the weight, the gradient-share mechanism is wrong; if it responds but saturation still cannot rescue every domain, insufficient physical loss is not the root cause.
4. **Is the injected slot load-bearing?** Tests 1–3 do not determine which channel prediction uses. We therefore apply *steering* (offset the position decoded from one channel and measure the response in the other), *counterfactual patching* (replace a channel with values from a donor trajectory and measure the resulting shift), and per-dimension Jacobians of predicted position. Norm-matched random directions and models without an injected slot serve as controls. A slot that carries no predictive load should have negligible effects in all three analyses.

3.4 The Training-Protocol Control: Free Rollout

Free rollout serves two methodological roles; neither is a novelty claim. It is the fully autoregressive endpoint of training against model-generated histories, closely related to scheduled sampling and multi-step prediction (Bengio et al. 2015; Venkatraman, Hebert, and Bagnell 2015).

The training protocol as a controlled factor. Physical structure has never been compared against the training protocol on the same baseline; prior studies treat the protocol as background. We first replace teacher forcing with free rollout. We then ask whether physical structure adds value beyond this protocol change. Teacher forcing hides compounding error at training time (*exposure bias*); free rollout supervises the full H-step rollout, exposing it and injecting no physics. The horizon H must match dynamics complexity: domains with late causal events need horizons long enough to contain them.

Positive control. A negative result invites two deflations—"your implementation is broken" or "your metrics cannot detect improvement." A protocol change producing large gains on identical code and metrics rules out both, pinning the failure on the injected structure itself.

3.5 External and Cross-Backbone Controls

Two controls probe our account from outside the LeWM backbone: one tests whether an architecture that *removes* the redundancy succeeds where injection fails, the other whether the conclusions survive a different encoder.

Extrinsic port. We port the physically interpretable world model (PIWM) of Mao, Umasudhan, and Ruchkin (2024)—a VAE encoder, supervised extraction of a low-dimensional physical state, and known-equation dynamics—to the same domains and OOD protocol. Because physical state is the sole input to its dynamics module, this architecture provides a control in which the shared-latent redundancy is absent.

Cross-backbone control. We replicate the core protocol on a second JEPA-family instantiation in the style of DINO-WM (Zhou et al. 2025): a *frozen* DINOv2-small encoder (Oquab et al. 2024) that has never seen PhyWorld, the trainable projector as adapter, and the identical predictor stack, losses, seeds, and evaluation—differing from LeWM in backbone provenance, dimensionality, and in whether the injected losses can reshape the encoder at all.

4 Experiments

4.1 Setup

Model. The compact backbone of §3.1 allows the complete factorial anatomy ($\sim$200 training runs). The 27-cell scan and its baselines, the training-regime 2×2, and the Physion++ injection table use three seeds ($3072/1234/42$). The re-weighting sweep is single-seed, with its two parity cells re-run over all three seeds.

Datasets and OOD protocol. All benchmarks are *visual* world-modeling tasks (RGB video), not the numerical/tabular sequences of prior physics-injection work (§2). Thus, injected state is always a small fraction of a high-dimensional visual latent. *PhyWorld* (Kang et al. 2024) has three single-mechanism domains: *uniform motion*, *parabola*, and *collision*, with zero, constant, and impulsive acceleration, respectively. Each domain contains $1{,}000$ ID trajectories. We evaluate four partitions: ID, r/m-OOD, v-OOD, and both-OOD. Their single-variable OOD ranges isolate physical extrapolation. Two *realistic-simulation* benchmarks (rendered, not camera video) add realism: *Physion* (Bear et al. 2021) for zero-shot object-contact prediction, and *Physion++* (Tung et al. 2023) (ten physical-property scenarios) for direct training with rollout to horizon 64.

Metrics and decision rules. We use two scale-aware, *external* metrics: latent nMSE ($\|\hat{\mathbf{z}}-\mathbf{z}\|^2$ divided by target variance) and pixel PSNR of decoded rollouts. They are external because no injection variant optimizes them directly. They agree in every case we examined. Latent cosine and probe decodability ρ (Alain and Bengio 2017) are *diagnostics only*: both are duals of the auxiliary losses and rise mechanically when those are optimized. nMSE has the opposite failure mode: its denominator degenerates when target variance collapses. We therefore judge parabola on its clean r/m-OOD partition (pathology catalogue: Appendix F of the supplement). Splits are trajectory-level; three-seed results report mean$\pm$sample std.

4.2 Physical Injection Fails to Improve Prediction

Figure 2 shows that physical injection degrades or fails to improve prediction in 26 of 27 mechanism–domain cells: 15 are worse in every seed and 11 remain within noise. Per-seed values are reported in Appendix B of the supplement. The hard slot learns position, reducing its supervision loss from 2.53 to 0.015 and increasing slot decodability from 0.31 to 0.96, but prediction remains unchanged or worsens by 1.16–$1.30\times$. Soft deep supervision, previously effective in an 8-D state latent (Zahorodnii 2025), yields 1.36–$1.37\times$ the baseline error on both-OOD domains in the 192-D visual latent;

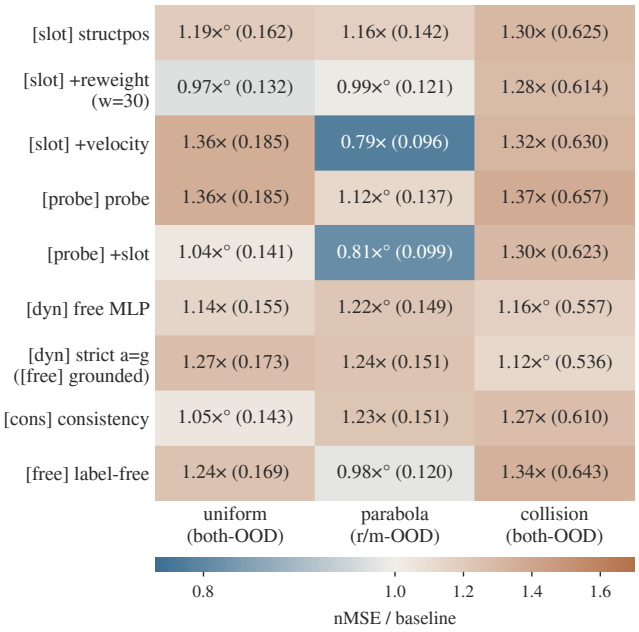


Figure 2: Every injection variant × domain vs. the free-rollout baseline (nMSE ratio; every cell and the baseline are three-seed means). Warm = worse, blue = better; ° = within noise (the cell's seeds overlap the baseline's). Injection fails to improve on baseline in 26 of 27 cells (15 worse, 11 within noise). The sole seed-robust gain is [slot]+velocity on parabola (0.79×). Strict $a{=}g$ and [free] grounded share one experimental configuration.

combining hard and soft injection does not improve it. The strict $a{=}g$ head increases uniform-motion error by 1.27× and performs similarly to learned acceleration on parabola (0.151 vs. 0.149). The collision-specific consistency loss also increases collision error by 1.27×. Finally, the label-free prior and its otherwise identical labeled control both underperform on collision (1.34× and 1.12×, respectively), so access to labels does not explain the result.

The only improvement occurs when velocity is added to the re-weighted slot on parabola ($0.122{\pm}0.007 \rightarrow 0.096{\pm}0.007$, lower in every seed). In this domain, velocity is the extrapolating variable. The same encoding is harmful on uniform motion, where velocity is constant, and on collision, where it changes discontinuously. Its effect therefore depends on prior knowledge of the domain dynamics, and its magnitude is substantially smaller than the training-protocol effect in §4.4.

From-scratch co-training. Table 2 reports latent nMSE on the hardest clean split for the strict $a{=}g$ head with a fixed slot; lower is better. Entries are three-seed means, Δ is injection on minus off, and ° marks overlap with the baseline's seed range. Introducing the constraint from the beginning of training does not change the overall result. Across three seeds, injection increases error on uniform motion (+0.435, Welch 95% CI $[+0.276, +0.594]$) and collision (+0.147, $[+0.074, +0.220]$). The parabola estimate is close to zero

Domain	Regime	phys. off	phys. on	Δ
uniform	scratch	0.222±0.028	0.657±0.095	**+0.435**
	fine-tune	0.136	0.173	+0.037
parabola (r/m-OOD)	scratch	0.476±0.169	0.447±0.063	−0.029
	fine-tune	0.122	0.151	+0.029
collision	scratch	0.502±0.043	0.648±0.013	**+0.147**
	fine-tune	0.479	0.536°	+0.057

Table 2: Injection on/off × training regime.

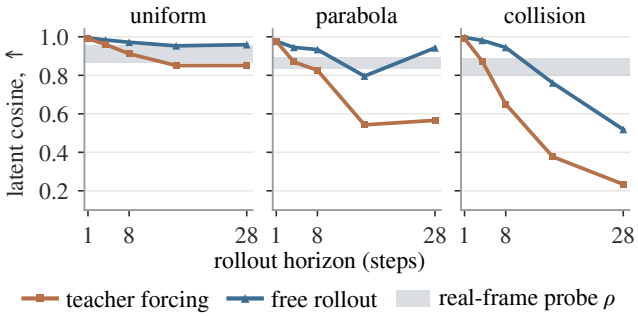


Figure 3: Decodable, yet drifting.

(−0.029, $[-0.318, +0.260]$) because the from-scratch baseline varies substantially across seeds. None of the three domains shows a resolved improvement, and the largest degradation in the grid occurs for from-scratch uniform motion (+0.435, compared with +0.037 after fine-tuning).

Two corroborations independent of PhyWorld nMSE. First, *zero-shot transfer*: on Physion OCP, no synthetic-trained configuration exceeds the random-encoder prior (0.607 mean AUC). The physics-structured variant is the *worst* configuration (0.551, below free rollout's 0.603). Thus, adding structure worsens transfer. Most apparent zero-shot competence comes from the architecture rather than learning (Appendix F of the supplement).

Second, *realistic simulation*: we train each ported injection variant directly on Physion++ for three seeds. The variants cover all three mechanism families: slot, consistency, and deep-supervision probe. Every variant degrades rollout nMSE by 3.6–11.9× relative to the identically trained free-rollout model. The degradation occurs in *every* seed on all three resolvable mass scenarios. The fourth scenario is too baseline-unstable to resolve (Appendix B of the supplement).

4.3 Mechanism: Load-Bearing but Redundant

In Figure 3, the gray bands report position decodability from real-frame embeddings on the both-OOD split, while the curves report three-seed mean latent cosine under teacher forcing and free rollout. Position remains linearly decodable at every horizon even as rollout similarity declines. Under teacher forcing, mean cosine falls from 0.98–0.99 at one step to 0.85 on uniform motion, 0.57 on parabola, and 0.23 on collision at horizon 28; free rollout recovers much of this gap without physics. Because decodability does not show whether prediction uses this information (Elazar et al. 2021),

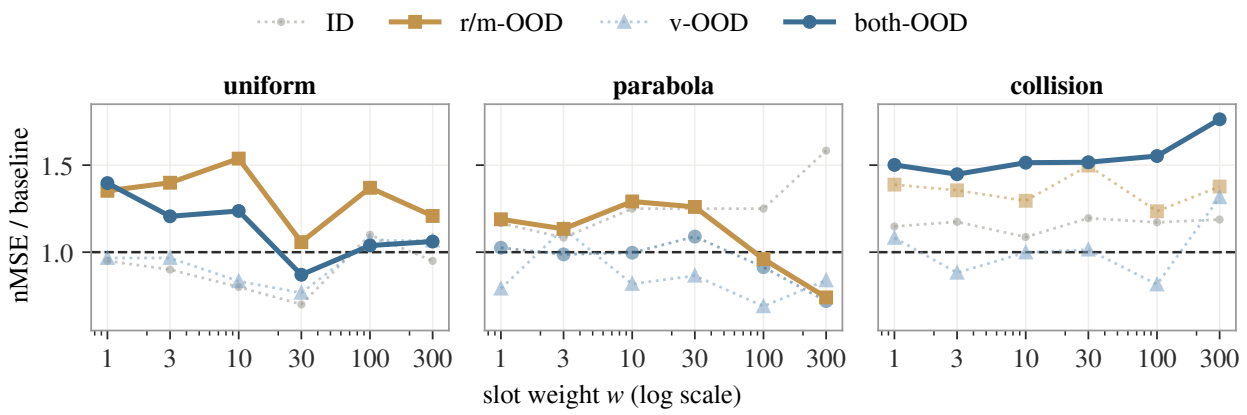


Figure 4: Dose–response to re-weighting across all partitions.

	uniform	parabola	collision
The redundant copy: position decodability ρ (both-OOD)			
full 192-d latent	0.92	0.85	0.78
black box only (190 d)	0.92	0.85	0.79
black box, slot pinned	0.92	0.86	0.79
the pinned slot itself (2 d)	0.92	0.85	0.51
random-2-d control		0.2–0.5	
The cost: what injection does to the encoder, λ=1→10			
loss ratio $(\lambda\,\mathcal{L}_{\text{phys}})/\mathcal{L}_{\text{pred}}$	11→44×	21→125×	3→15×
effective dim	41→4	13→3	28→17
collapse	−90%	−77%	−39%

Table 3: Redundancy and optimization effects. *Top*: position decodability ρ on both-OOD. *Bottom*: physical-to-prediction loss ratio and encoder effective dimension as λ increases from 1 to 10.

we apply the four tests defined in §3.3. Table 3 summarizes two relevant observations: the non-slot dimensions retain position information (top), while the injection loss can dominate optimization and reduce encoder dimensionality (bottom).

Test 1: the physical constraint is injected but redundant. The top half of Table 3 shows that probing *only the* 190 *non-slot dimensions* decodes position as well as the full latent in all three domains, undiminished by pinning the slot, while a random-2-dimension control fails. Thus, the position is injected but a redundant copy.

Test 2: the redundant constraint acts harmfully. Both measurements in the bottom half of Table 3 move: at λ=10 the weighted physical loss outweighs the prediction loss by one to two orders of magnitude, and the encoder's effective dimensionality collapses with it; at λ=1, the setting of §4.2, the ratio is milder (3–21×) yet the harm still persists.

Test 3: re-weighting the physical loss does not help. Figure 4 traces the single-seed sweep from w=1 to 300, with lower nMSE indicating better performance. Solid curves mark the four decision cells; dotted curves mark the non-decision partitions (ID and the denominator-degenerate parabola OOD partition; Appendix F of the supplement). Weighting moves the harm systematically but rescues only 2 of 4 decision cells, to *parity* and never a net gain: uniform both-OOD returns exactly to baseline (0.132±0.017 vs. 0.136±0.008), uniform r/m-OOD never recovers, and collision worsens monotonically. The two parity cells were re-run over three seeds; detailed values are provided in Appendix B of the supplement. Thus, insufficient physical loss is not the root cause.

Test 4: the injected slot is load-bearing. We test the slot's causal role three disjoint ways—two interventions and a non-interventional check—to examine whether the injected slot is load-bearing (three seeds, both backbones; excluded cells and full protocol: Appendix D of the supplement). *Steering* the position the slot decodes moves the model's own black-box state by 1.5–2.0× the offset, while a norm-matched random direction moves it ≈0 (additive to within 3%, so the perturbation stays linear). *Patching* the slot with a donor trajectory's makes prediction follow the donor 47–72% of the way, against 1.5–11% for the same operation on a model with no injected slot. *Jacobians* perturb nothing. They rank the two injected dimensions first and second out of 192 for predicted position, compared with about 99th without injection. All three analyses agree: the injected slot is load-bearing. Prediction reads it and depends on it causally.

The four tests support the load-bearing but redundant account. The slot duplicates state retained in the non-slot dimensions (Test 1), affects capacity and optimization without adding a measurable predictive advantage (Tests 2–3), and lies on the prediction pathway (Test 4).

4.4 Free Rollout Improves Long-Horizon Prediction Accuracy

Figure 5 reports nMSE on the hardest clean split as three-seed mean±sample std; lower is better. Solid bars denote LeWM and hatched bars denote frozen DINOv2. In the axis labels, r/m means radius/mass-OOD, sim. means realistic simulation, and h64 means evaluation at horizon 64. Replacing teacher forcing with free rollout—a training-protocol change that injects no physics—cuts the hardest-split error 2.2–3.6×

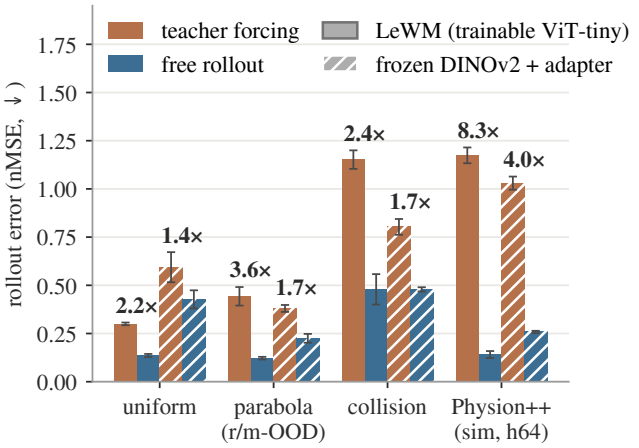


Figure 5: Teacher forcing versus free rollout across backbones and domains.

across the three domains and $8.3\times$ on Physion++, with every interval disjoint over three seeds. The whole pattern replicates on frozen DINOv2; full per-bar values are reported in Appendix C of the supplement. The gain is not an OOD patch: *every* partition improves, including ID (2.0–$4.6\times$). The horizon must match dynamics complexity—collision improves markedly at $H{\approx}20$ while smooth domains degrade with excess horizon, and realistic-simulation scenes benefit most from a longer horizon: $H{=}8 \rightarrow 28$ drives horizon-64 nMSE from 0.280 to 0.014, $\sim 19\times$ (Appendix E of the supplement).

This experiment serves as the positive control described in §3.4. Using the same code, metrics, seeds, and data as the injection scan, a change in training protocol produces 2.2–$8.3\times$ gains. The null injection results therefore cannot be attributed to an evaluation pipeline that is insensitive to improvement.

4.5 External Control: Removing the Redundancy Is Not Enough

Extrinsic port. The faithful extrinsic port (PIWM; §3.5) learns the correct physics. When its encoder is in-distribution, it slightly outperforms our shared-latent model (ρ 0.96/0.97 vs. 0.93/0.87). Under radius/mass shift, however, its VAE encoder faces an appearance shift. PIWM then falls to $\rho{=}0.33$/0.48, while LeWM remains at 0.89/0.87.

This result supports the account in §4.3. A sole, non-redundant physical state can provide a small in-distribution gain, whereas injecting the same equations into a shared latent cannot (§4.2). However, PIWM concentrates prediction in one brittle encoder. Removing redundancy therefore trades the duplicated-state problem for an encoder-OOD problem rather than solving it. An extrinsic sole-carrier design is not, by itself, a fix.

Cross-backbone replication. The frozen-DINOv2 experiments reproduce the primary findings over three seeds. The encoder decodes position at $\rho{=}0.95$ (compared with 0.90 for LeWM), and its non-slot dimensions retain the same information, indicating that redundancy is not specific to LeWM training. Figure 5 also shows a 1.4–$4.0\times$ improvement from free-rollout training. Injection fails to improve 26 of 27 cells; the remaining cell reaches $0.97\times$ baseline with a 0.001 nMSE difference that lies within seed noise. Content-free controls attribute the apparent uniform-motion gains to capacity-restriction regularization rather than physics (Trap 5, Appendix F of the supplement). With the encoder frozen, injection is mostly inert rather than harmful because its losses can reshape only the adapter. These results reproduce the redundancy and training-protocol effects with a different visual backbone.

5 Conclusion

We evaluated nine forms of physical injection in a shared-latent world model across three controlled domains and two realistic simulation benchmarks. Injection fails to improve prediction in 26 of 27 mechanism–domain combinations, and the result transfers from LeWM to a frozen DINOv2 backbone. The non-slot dimensions retain the same physical information as the injected slot, while interventions show that prediction nevertheless depends on the slot. This combination makes the slot load-bearing but redundant: an incorrect constraint can degrade prediction, whereas a correct one contributes little new information. An extrinsic architecture that removes this redundancy improves in-distribution performance but remains sensitive to OOD shifts in its encoder.

Free-rollout training reduces error by 2.2–$8.3\times$ under the same code, metrics, and seeds, indicating that long-horizon state evolution is a more consequential limitation than the availability of physical state. More generally, probe decodability does not imply that explicitly injecting the decoded variable will improve prediction, particularly when the representation already contains redundant copies of that information.

Limitations and Future Work. Our conclusions are limited to the three injection axes studied in a shared high-dimensional latent. They do not directly cover reconstruction-based models such as Dreamer or architectures that make physical state the only input to the transition model. The variants were selected to cover the design space rather than exhaustively optimize any one mechanism. The favorable velocity+slot result on parabola suggests that dynamics-matched injection may still help when the relevant transition variable is known.

References

Alain, G.; and Bengio, Y. 2017. Understanding Intermediate Layers Using Linear Classifier Probes. In *5th International Conference on Learning Representations (ICLR 2017), Workshop Track Proceedings*. OpenReview.net.

Assran, M.; Duval, Q.; Misra, I.; Bojanowski, P.; Vincent, P.; Rabbat, M.; LeCun, Y.; and Ballas, N. 2023. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 15619–15629.

Balestriero, R.; and LeCun, Y. 2025. Lejepa: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*.

Bardes, A.; Garrido, Q.; Ponce, J.; Chen, X.; Rabbat, M.; LeCun, Y.; Assran, M.; and Ballas, N. 2024. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*.

Battaglia, P.; Pascanu, R.; Lai, M.; Jimenez Rezende, D.; and kavukcuoglu, k. 2016. Interaction Networks for Learning about Objects, Relations and Physics. In Lee, D.; Sugiyama, M.; Luxburg, U.; Guyon, I.; and Garnett, R., eds., *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.

Bear, D. M.; Wang, E.; Mrowca, D.; Binder, F. J.; Tung, H.-Y. F.; Pramod, R.; Holdaway, C.; Tao, S.; Smith, K.; Sun, F.-Y.; et al. 2021. Physion: Evaluating physical prediction from vision in humans and machines. *arXiv preprint arXiv:2106.08261*.

Bengio, S.; Vinyals, O.; Jaitly, N.; and Shazeer, N. 2015. Scheduled sampling for sequence prediction with recurrent neural networks. *Advances in neural information processing systems*, 28.

Cranmer, M.; Greydanus, S.; Hoyer, S.; Battaglia, P.; Spergel, D.; and Ho, S. 2020. Lagrangian neural networks. *arXiv preprint arXiv:2003.04630*.

Elazar, Y.; Ravfogel, S.; Jacovi, A.; and Goldberg, Y. 2021. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9: 160–175.

Greydanus, S.; Dzamba, M.; and Yosinski, J. 2019. Hamiltonian neural networks. *Advances in neural information processing systems*, 32.

Ha, D.; and Schmidhuber, J. 2018. Recurrent world models facilitate policy evolution. *Advances in neural information processing systems*, 31.

Hafner, D.; Pasukonis, J.; Ba, J.; and Lillicrap, T. 2025. Mastering diverse control tasks through world models. *Nature*, 640(8059): 647–653.

Hewitt, J.; and Liang, P. 2019. Designing and Interpreting Probes with Control Tasks. In Inui, K.; Jiang, J.; Ng, V.; and Wan, X., eds., *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2733–2743. Hong Kong, China: Association for Computational Linguistics.

Kang, B.; Yue, Y.; Lu, R.; Lin, Z.; Zhao, Y.; Wang, K.; Huang, G.; and Feng, J. 2024. How far is video generation from world model: A physical law perspective. *arXiv preprint arXiv:2411.02385*.

Lamb, A. M.; ALIAS PARTH GOYAL, A. G.; Zhang, Y.; Zhang, S.; Courville, A. C.; and Bengio, Y. 2016. Professor forcing: A new algorithm for training recurrent networks. *Advances in neural information processing systems*, 29.

LeCun, Y.; et al. 2022. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1): 1–62.

Liu, Z.; Sanborn, S.; Ganguli, S.; and Tolias, A. 2026. From Kepler to Newton: Inductive Biases Guide Learned World Models in Transformers. *arXiv preprint arXiv:2602.06923*.

Locatello, F.; Weissenborn, D.; Unterthiner, T.; Mahendran, A.; Heigold, G.; Uszkoreit, J.; Dosovitskiy, A.; and Kipf, T. 2020. Object-Centric Learning with Slot Attention. In Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.; and Lin, H., eds., *Advances in Neural Information Processing Systems*, volume 33, 11525–11538. Curran Associates, Inc.

Maes, L.; Lidec, Q. L.; Scieur, D.; LeCun, Y.; and Balestriero, R. 2026. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*.

Mao, Z.; Umasudhan, M. E.; and Ruchkin, I. 2024. Physically Interpretable World Models via Weakly Supervised Representation Learning. *arXiv preprint arXiv:2412.12870*.

Nam, H.; Lidec, Q. L.; Maes, L.; LeCun, Y.; and Balestriero, R. 2026. Causal-JEPA: Learning World Models through Object-Level Latent Masking. In *International Conference on Machine Learning (ICML)*.

Nie, W.; Liu, W.; Guo, H.; and Su, Y. 2026. Phys-JEPA: Physics-Informed Latent World Models for Multivariate Time-Series Forecasting. *arXiv preprint arXiv:2606.16076*.

Oquab, M.; Darcet, T.; Moutakanni, T.; Vo, H.; Szafraniec, M.; Khalidov, V.; Fernandez, P.; Haziza, D.; Massa, F.; El-Nouby, A.; et al. 2024. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research Journal*.

Peper, J.; Mao, Z.; Geng, Y.; Pan, S.; and Ruchkin, I. 2025. Four principles for physically interpretable world models. *arXiv preprint arXiv:2503.02143*.

Raissi, M.; Perdikaris, P.; and Karniadakis, G. E. 2018. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378(C).

Sanchez-Gonzalez, A.; Godwin, J.; Pfaff, T.; Ying, R.; Leskovec, J.; and Battaglia, P. 2020. Learning to Simulate Complex Physics with Graph Networks. In III, H. D.; and Singh, A., eds., *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, 8459–8468. PMLR.

Tung, H.-Y.; Ding, M.; Chen, Z.; Bear, D.; Gan, C.; Tenenbaum, J. B.; Yamins, D. L.; Fan, J. E.; and Smith, K. A. 2023. Physion++: Evaluating physical scene understanding that requires online inference of different physical properties. *arXiv preprint arXiv:2306.15668*.

Venkatraman, A.; Hebert, M.; and Bagnell, J. 2015. Improving multi-step prediction of learned time series models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29.

Zahorodnii, A. 2025. Improving World Models using Deep Supervision with Linear Probes. *arXiv preprint arXiv:2504.03861*.

Zhou, G.; Pan, H.; Lecun, Y.; and Pinto, L. 2025. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 79115–79135. PMLR.